Java Core Модуль 1

От исходного текста до работающей программы: Linux, Vim, Git, модель исполнения и точные вычисления.

Практический учебник на русском языке. Для читателя, знакомого с базовым синтаксисом Java, Python и основами C/C++. Редакция 9 ноября 2026 года.

Объём: 5 недель по 15-18 часов, всего около 80 часов. Это первый модуль, а не весь Java Core. Здесь закладывается фундамент: инструменты, семантика значений и ссылок, численные алгоритмы, контракты, диагностика и самостоятельная сборка.

Как учиться

На каждое занятие: 60-90 минут чтения с прогнозированием вывода; 2 часа воспроизведения примеров вручную в Vim; 2-3 часа задач; 1 час отчёта и исправлений. Не копируйте программу целиком: после каждого метода компилируйте и объясняйте результат. Сначала запишите ожидаемый вывод, потом запускайте.

Базовая среда учебника: Linux, Bash и JDK 21 или новее с компиляцией --release 21. Никаких preview-функций, IDE и систем сборки для обязательной части. В Windows используйте отдельную Linux-среду, например WSL; все команды ниже выполняются внутри Linux. Установка Linux не является предметом этого модуля.

Маршрут

1. Linux, потоки процесса, Vim и первый коммит (8 ч).

2. javac, java, classpath и байткод (8 ч).

3. Целые числа, биты и доказательства границ (9 ч).

4. Методы, ссылки и массивы (8 ч).

5. Численный анализ и строки Unicode (9 ч).

6. Классы, инварианты и исключения (9 ч).

7. Тестирование, jdb и расследование ошибки (8 ч).

8. JAR, воспроизводимость и Git (6 ч).

Итоговая лабораторная и защита (15 ч).

Занятие 1. Linux как рабочая среда

Программа существует внутри процесса. У процесса есть текущий каталог, окружение, аргументы, код завершения и три стандартных потока: stdin, stdout, stderr. Файл с исходниками не становится процессом: сначала оболочка запускает java, затем JVM загружает класс и вызывает main. Текущий каталог влияет на относительные пути, но не определяет пакет класса.

Путь /home/student/java/module1 абсолютный; src/Main.java относительный. Символ ~ раскрывается оболочкой в домашний каталог. Кавычки сохраняют пробелы внутри одного аргумента. Linux различает регистр: Main.java и main.java являются разными именами. Команда pwd показывает каталог процесса оболочки, а cd меняет его для следующих команд.

```
sudo apt update
sudo apt install openjdk-21-jdk vim git
java -version
javac -version
command -v java javac javap jar jdb
mkdir -p ~/java/module1/src ~/java/module1/out
cd ~/java/module1
pwd
ls -la
vim src/Hello.java
```

Команды установки приведены для Debian/Ubuntu с доступным пакетом JDK 21. Если пакет недоступен в вашем выпуске, установите JDK 21 из пакетов своего дистрибутива. Проверяйте версии java и javac: случайная смесь двух установок приводит к ошибкам совместимости.

Vim: редактирование как язык действий

Vim запускается в normal mode. Нажмите i, введите текст, Esc, затем :w для сохранения и :q для выхода. :wq делает оба действия, :q! отбрасывает несохранённые изменения. Движения h j k l соответствуют влево, вниз, вверх, вправо; w переходит к следующему слову, 0 к началу строки, $ к концу, gg к началу файла, G к концу.

Оператор действует на движение: dw удаляет до следующего слова, ciw заменяет слово, yy копирует строку, p вставляет, u отменяет, Ctrl-r повторяет отменённое. /main ищет текст, n повторяет поиск. :set number включает номера, :set expandtab shiftwidth=4 softtabstop=4 задаёт отступы. :%s/old/new/g заменяет все совпадения; добавьте c в конце для подтверждения каждой замены. Освойте :help и пройдите vimtutor.

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java");
        System.err.println("diagnostic");
    }
}

javac -encoding UTF-8 -d out src/Hello.java
java -cp out Hello > result.txt 2> error.txt
status=$?
printf "exit=%s\n" "$status"
cat result.txt
cat error.txt
```

println добавляет перевод строки. Первый вызов пишет в stdout, второй в stderr. Перенаправление > создаёт или перезаписывает файл, 2> перенаправляет дескриптор 2. Код $? нужно сохранить сразу: следующая команда его заменит. Ожидаются Hello, Java в result.txt, diagnostic в error.txt и

exit=0. Конвейер | соединяет stdout одного процесса со stdin следующего; stderr автоматически в него не попадает.

Git с первого дня

```
git init
git config user.name "Student"
git config user.email "student@example.invalid"
printf "out/\n*.class\n*.jar\n" > .gitignore
git add .gitignore src/Hello.java
git diff --cached
git commit -m "Add first compiled program"
git status
git log --oneline
```

Рабочее дерево содержит текущие файлы, индекс содержит снимок следующего коммита, коммит хранит снимок проекта и ссылку на родителя. git add не подписывается на будущие изменения файла: после нового редактирования добавьте его снова. git diff показывает изменения относительно индекса, --cached сравнивает индекс с HEAD. Игнорирование не снимает с отслеживания уже добавленный файл.

Практика и самопроверка

1.1. База: воспроизведите Hello и разделите потоки в файлы.

1.2. Средний: создайте каталог с пробелом в имени и соберите программу из него. Объясните роль кавычек.

1.3. Сложный: измените программу после git add и покажите два разных diff. Предскажите содержимое следующего коммита.

Контрольные вопросы: Чем stdout отличается от stderr? Почему cd выполняется оболочкой? Что именно сохраняет git add?

Лабораторная работа: Создайте репозиторий, README с версией JDK и журналом команд. Сделайте два содержательных коммита, второй - исправление намеренной ошибки компиляции.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Занятие 2. Компиляция и модель JVM

javac проверяет синтаксис, типы, доступность имён и создаёт class-файлы. JVM загружает классы, проверяет байткод и исполняет его; реализация может интерпретировать методы и компилировать горячие участки в машинные инструкции. class-файл не является машинным кодом конкретного процессора. Переносимость ограничена версией формата, доступными API и платформенными зависимостями.

JDK содержит инструменты разработки и среду исполнения. JVM - виртуальная машина, а не синоним всего JDK. Java автоматически управляет памятью объектов, но не закрывает произвольные файловые дескрипторы по окончании области видимости. Устройство сборщика мусора не следует выводить из поведения одной маленькой программы.

```
mkdir -p src/course
vim src/course/Main.java

package course;
public class Main {
    public static void main(String[] args) {
        int x = 7;
        System.out.println(x * 6);
    }
}

javac --release 21 -encoding UTF-8 -g -Xlint:all \
  -d out src/course/Main.java
java -cp out course.Main
javap -classpath out -c -l -p course.Main
javap -classpath out -verbose course.Main
```

Имя запуска course.Main - двоичное имя класса, не путь к исходнику. Корень classpath out должен содержать каталог course. Опция -d создаёт структуру пакетов. --release 21 ограничивает язык, формат class и доступные API целевой платформой; это надёжнее одной пары -source/-target. -g добавляет сведения для отладки. -Xlint:all включает дополнительные предупреждения, но не доказывает корректность.

В байткоде метода main найдите загрузку констант, сохранение локальной переменной и вызов println. imul потребляет два int со стека операндов и оставляет произведение. Локальные переменные кадра метода и стек операндов - части абстрактной JVM, не прямое описание регистров процессора. Таблица номеров строк связывает инструкции с исходником; -g помогает отладчику показывать локальные имена.

ClassNotFoundException при запуске обычно указывает на имя или classpath; ошибка компиляции cannot find symbol возникает раньше. UnsupportedClassVersionError означает, что среда исполнения не понимает версию class-файла. NoSuchMethodError может появиться при несовместимых версиях классов: успешная компиляция не гарантирует согласованность библиотек при запуске.

Практика и самопроверка

2.1. База: получите 42 и найдите imul в javap.

2.2. Средний: запустите из родительского каталога с корректным относительным classpath. Затем воспроизведите ошибку поиска класса.

2.3. Сложный: сравните байткод x * 6 и константы 7 * 6. Объясните свёртку констант без предположений о JIT.

Контрольные вопросы: Почему classpath указывает на корень пакетов? Чем javap отличается от декомпилятора исходников? Что меняет -g?

Лабораторная работа: Составьте карточки трёх ошибок: команда, сообщение, причина, исправление. Для каждой изменяйте только один фактор.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Занятие 3. Целые числа и биты

byte, short, int, long - знаковые целые шириной 8, 16, 32, 64 бита. Для n бит диапазон равен [-2^(n-1), 2^(n-1)-1]. char - беззнаковая 16-битная кодовая единица UTF-16. boolean не является целым числом. В отличие от Python, int не растёт автоматически. Операции int выполняются с отбрасыванием старших битов: переполнение обычного сложения не вызывает исключения.

Большинство арифметических операций над byte и short сначала продвигает операнды до int. Присваивание byte c = a + b не компилируется без приведения; c += b содержит неявное сужение. Приведение (long)(a * b) слишком поздно, если оба операнда int: переполнение уже произошло. Нужно (long)a * b. Для операций с обязательной проверкой границ используйте Math.addExact и Math.multiplyExact.

```
public class IntegerLab {
    public static void main(String[] args) {
        int a = 50_000;
        System.out.println(a * a);
        System.out.println((long) a * a);
        System.out.println(-7 / 3);
        System.out.println(-7 % 3);
        System.out.println(-1 >>> 1);
    }
}
```

Вывод: -1794967296, 2500000000, -2, -1, 2147483647. Деление целых округляет к нулю. Остаток имеет знак делимого или равен нулю; при ненулевом b верно a = (a / b) * b + a % b в арифметике типа. Деление на ноль бросает ArithmeticException. Особый случай MIN_VALUE / -1 возвращает MIN_VALUE из-за переполнения.

Дополнительный код удобен тем, что одно двоичное сложение работает для положительных и отрицательных значений. Отрицание x эквивалентно ~x + 1 по модулю 2^n. Поэтому у MIN_VALUE нет положительного аналога того же типа, а Math.abs(MIN_VALUE) остаётся отрицательным. Это контрпример к наивной реализации алгоритма Евклида через abs(int).

Операторы & | ^ ~ работают по битам; && и || работают с boolean и сокращённым вычислением. >> распространяет знаковый бит, >>> заполняет нулями. Для int расстояние сдвига маскируется до 5 младших битов: 1 << 32 равно 1, а не 0. Для long используются 6 бит. Проверка одного бита: (x & (1 << k)) != 0 при заранее проверенном k от 0 до 31.

Разбор алгоритма Евклида

```
public class Gcd {
    static long gcd(int x, int y) {
        long a = Math.abs((long) x);
        long b = Math.abs((long) y);
        while (b != 0) {
            long r = a % b;
            a = b;
            b = r;
        }
        return a;
    }
    public static void main(String[] args) {
        System.out.println(gcd(Integer.MIN_VALUE, 0));
        System.out.println(gcd(48, 18));
    }
}
```

Получаем 2147483648 и 6. Приведение к long предшествует abs. Инвариант: множество общих делителей a и b сохраняется при переходе (a,b) -> (b,a%b). При b > 0 остаток строго меньше b, поэтому неотрицательная целочисленная мера убывает и цикл завершается. В конце gcd(a,0)=a. По контракту gcd(0,0)=0; это программное соглашение. В модели арифметических шагов число итераций O(log max(a,b)); стоимость деления больших чисел требует отдельной модели.

Практика и самопроверка

3.1. База: предскажите результат для MAX_VALUE + 1, (byte)128 и 1 << 31. Проверьте.

3.2. Средний: реализуйте lcm для int с результатом long, сокращая через gcd до умножения. Задайте контракт для нулей.

3.3. Сложный: докажите границы результата lcm для int, включая MIN_VALUE; сравните с BigInteger в отдельном проверочном коде.

Контрольные вопросы: Почему abs не всегда положителен? Когда приведение к long запоздало? Почему Евклид завершается?

Лабораторная работа: Создайте числовую библиотеку gcd/lcm и таблицу минимум из 12 случаев: нули, знаки, взаимно простые числа, MIN/MAX. Для каждой строки укажите математически ожидаемый ответ.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Занятие 4. Методы, значения и ссылки

Java всегда передаёт аргументы по значению. Для примитива копируется само значение; для объекта копируется значение ссылки. Метод может изменить доступный через ссылку массив, но присваивание параметру новой ссылки не меняет переменную вызывающего метода. Аналогия с C++: параметр Java не становится T&. Аналогия с Python полезна для совместного доступа к объекту, но не заменяет статическую систему типов Java.

```
public class References {
    static void change(int[] a) {
        a[0] = 9;
        a = new int[] {100};
    }
    static int lowerBound(int[] a, int key) {
        int left = 0, right = a.length;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (a[mid] < key) left = mid + 1;
            else right = mid;
        }
        return left;
    }
    public static void main(String[] args) {
        int[] values = {1, 3, 3, 8};
        System.out.println(lowerBound(values, 3));
        change(values);
        System.out.println(values[0]);
    }
}
```

Вывод 1 и 9. После change массив уже не отсортирован, поэтому повторный бинарный поиск нарушил бы предусловие. Переназначение a создало новый массив и изменило только локальный параметр. Ссылки сравниваются == по идентичности; содержимое массивов сравнивают Arrays.equals. null означает отсутствие ссылки; обращение к элементу через null приводит к NullPointerException.

Массив имеет фиксированную длину. new int[n] заполняет элементы нулями, массив ссылочного типа - null. Локальная переменная сама по себе не получает автоматически допустимое значение для чтения: компилятор требует definite assignment. Индексы от 0 включительно до length исключительно. Двумерный int[][] является массивом ссылок на массивы, строки могут иметь разную длину.

Доказательство lowerBound

Контракт: входной массив отсортирован неубывающе; результат - первый индекс i с a[i] >= key, либо length. Инвариант: 0 <= left <= right <= n, все элементы до left меньше key, все элементы начиная с right не меньше key. Изначально обе зоны пусты. Если a[mid] < key, вся левая часть до mid включительно исключается по сортировке. Иначе mid остаётся кандидатом, поэтому right=mid. Мера right-left строго уменьшается. При равенстве границ зоны встречаются в искомой позиции.

Формула mid избегает переполнения left+right при допустимых индексах. Алгоритм требует O(log n) сравнений и O(1) дополнительной памяти. Но проверка сортировки перед каждым запросом стоит O(n); это часть контракта, а не бесплатная операция. Рекурсивный вариант использовал бы O(log n) кадров стека.

Практика и самопроверка

4.1. База: нарисуйте ссылки до и после change; замените == на Arrays.equals в эксперименте с двумя массивами.

4.2. Средний: реализуйте upperBound и число вхождений ключа через две границы. Проверьте пустой массив и повторы.

4.3. Сложный: докажите корректность поиска первого true в монотонном предикате на целых индексах. Укажите, что сломается без монотонности.

Контрольные вопросы: Что именно копируется при вызове метода? Почему right равен length? В чём разница между контрактом и проверкой контракта?

Лабораторная работа: Сравните линейный поиск границы и бинарный на всех отсортированных массивах длиной до 6 над значениями {-1,0,1}. Генератор проверок должен быть самостоятельной частью отчёта.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Занятие 5. Вещественные числа и текст

double хранит приближение в двоичной системе IEEE 754. Многие десятичные дроби, включая 0.1, не представимы точно. Величина ошибки зависит от масштаба: расстояние между соседними представимыми числами меняется с порядком. Равенство double проверяет равенство представленных значений, а не математических выражений. Вычитание близких приближений может уничтожить значащие цифры.

```
public class Floating {
    static boolean close(double a, double b,
                         double abs, double rel) {
        if (abs < 0 || rel < 0 ||
            !Double.isFinite(abs) || !Double.isFinite(rel)) {
            throw new IllegalArgumentException("tolerance");
        }
        if (a == b) return true;
        if (!Double.isFinite(a) || !Double.isFinite(b)) return false;
        return Math.abs(a - b) <= Math.max(abs,
            rel * Math.max(Math.abs(a), Math.abs(b)));
    }
    public static void main(String[] args) {
        System.out.println(0.1 + 0.2 == 0.3);
        System.out.println(close(0.1 + 0.2, 0.3, 1e-15, 1e-12));
        System.out.println(Double.NaN == Double.NaN);
    }
}
```

Вывод false, true, false. Абсолютный допуск нужен около нуля, относительный - на больших масштабах. Допуски выводятся из требований задачи, а не выбираются по привычке. Пример close ориентирован на умеренные масштабы: разность и произведение rel*scale тоже могут переполниться при экстремальных конечных значениях. Приближённое равенство нетранзитивно и не подходит для equals ключей коллекции.

Деление double на ноль даёт бесконечность или NaN, а не целочисленное ArithmeticException. NaN не равен самому себе, используйте Double.isNaN. Денежная десятичная арифметика обычно требует BigDecimal из строки, например new BigDecimal("0.1"); конструктор из double переносит двоичную погрешность. Деление BigDecimal может потребовать явной точности и режима округления.

Бисекция как численная лабораторная

Для непрерывной функции f на [a,b] с противоположными знаками на концах теорема о промежуточном значении гарантирует корень. На каждом шаге выбирают половину с изменением знака. После k шагов длина интервала равна исходной длине / 2^k; для ошибки середины не более eps достаточно длины <= 2*eps. Условие малого |f(x)| само по себе не гарантирует малой ошибки x, если производная близка к нулю.

В машинной арифметике середина может совпасть с одним из концов: дальнейшее сужение невозможно. Поэтому реализация обязана иметь лимит итераций, проверку конечности значений и проверку mid==a или mid==b. Для ограниченного интервала [0,2] вычисляйте mid=a+(b-a)/2; для произвольных огромных концов отдельно анализируйте переполнение разности. Не проверяйте знак через f(a)*f(b): произведение может переполниться или округлиться к нулю.

Строки: кодовые единицы и символы

String неизменяем. length() считает кодовые единицы UTF-16; символ вне BMP занимает пару surrogate. Для строки "A□" length равно 3, а codePointCount(0,length) равно 2. И это ещё не число пользовательских графем: буква с комбинируемым акцентом может состоять из нескольких code point. charAt не обещает целый Unicode-символ. Для обхода кодовых точек используйте codePoints().

```
String s = "A ";
System.out.println(s.length());
System.out.println(s.codePointCount(0, s.length()));
s.codePoints().forEach(System.out::println);
```

Строки сравнивают equals; == проверяет идентичность объектов и может обманчиво работать для интернированных литералов. Конкатенация в цикле создаёт промежуточные строки; для накопления используйте StringBuilder. Кодировка файла UTF-8 и внутреннее представление текста на уровне API - разные вопросы. Явное -encoding UTF-8 устраняет зависимость компилятора от настроек среды.

Практика и самопроверка

5.1. База: получите false для 0.1+0.2==0.3 и объясните без фразы «Java ошибается».

5.2. Средний: реализуйте бисекцию x*x-2 на [0,2], сохраняя интервал и количество итераций. Сравните с Math.sqrt(2).

5.3. Сложный: исследуйте суммирование 1e16, 1, -1e16 в разных порядках; реализуйте компенсированное суммирование и опишите ограничения.

Контрольные вопросы: Почему eps не универсален? Чем code point отличается от char? Почему малая невязка не всегда означает малую ошибку аргумента?

Лабораторная работа: Создайте отчёт о бисекции: доказательство инварианта, теоретическое число шагов, фактический интервал, остановка при недостижимом eps, проверки NaN и бесконечностей.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Занятие 6. Классы и контракты

Класс связывает состояние и допустимые операции. Инвариант - свойство каждого корректного наблюдаемого состояния. Конструктор устанавливает инвариант; каждый открытый метод обязан его сохранять. private ограничивает доступ к представлению. final у поля запрещает переназначение, но не делает объект по ссылке неизменяемым. Для изменяемых массивов нужна защитная копия при приёме и выдаче.

```
public final class Fraction {
    private final long numerator;
    private final long denominator;

    public Fraction(int n, int d) {
        if (d == 0) throw new IllegalArgumentException("zero denominator");
        long a = n, b = d;
        if (b < 0) { a = -a; b = -b; }
        long x = Math.abs(a), y = b;
        while (y != 0) {
            long r = x % y;
            x = y; y = r;
        }
        numerator = a / x;
        denominator = b / x;
    }
    public long numerator() { return numerator; }
    public long denominator() { return denominator; }
    @Override public String toString() {
        return numerator + "/" + denominator;
    }
    public static void main(String[] args) {
        System.out.println(new Fraction(6, -8));
        System.out.println(new Fraction(0, -5));
    }
}
```

Вывод -3/4 и 0/1. Инварианты: знаменатель положителен, gcd(|n|,d)=1, ноль представлен как 0/1. Вход ограничен int, поэтому временные long безопасно вмещают отрицание MIN_VALUE. Перенос конструктора на long без пересмотра доказательства нарушит безопасность abs и отрицания. Поля final и отсутствие мутаций делают этот объект неизменяемым на уровне его состояния.

Исключение разрывает обычное выполнение и передаёт управление ближайшему подходящему catch. IllegalArgumentException сообщает о нарушении контракта вызывающим кодом. IOException - проверяемое исключение: его нужно обработать или объявить throws. Не ловите Exception только ради продолжения после неизвестной ошибки. На границе CLI ожидаемую ошибку ввода следует превратить в понятное сообщение stderr и ненулевой код завершения.

equals должен быть рефлексивным, симметричным, транзитивным и согласованным; для null возвращать false. Равные объекты обязаны иметь одинаковый hashCode. Fraction пока наследует идентичность Object: две равные математические дроби не равны через equals. Это осознанное ограничение примера, которое предстоит исправить в задании. toString отвечает за представление, а не за равенство.

Практика и самопроверка

6.1. База: проверьте Fraction(0,1), Fraction(2,4), Fraction(MIN_VALUE,-1), нулевой знаменатель.

6.2. Средний: добавьте equals и hashCode по нормализованным полям. Проверьте контракт на тройках дробей.

6.3. Сложный: спроектируйте сложение дробей с явной политикой переполнения: checked long или BigInteger. Не используйте молчаливое усечение.

Контрольные вопросы: Почему final-ссылка на массив не делает его неизменяемым? Как доказать нормализацию нуля? Что случится при изменении типа входа на long?

Лабораторная работа: Реализуйте неизменяемый диапазон целых с проверяемыми границами. Опишите включённость концов, пустоту, пересечение и безопасное вычисление длины в long.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Занятие 7. Проверки и отладчик

Тест - исполняемая проверка конкретного свойства, а не доказательство для всех входов. Разделяйте примерные тесты, граничные случаи и свойства. Для gcd полезны симметрия, неотрицательность и делимость исходных значений результатом при ненулевом результате. Свойство gcd(a,b)<=min(abs(a),abs(b)) неверно при одном нулевом аргументе: даже тестовое свойство нуждается в предусловии.

Встроенный assert отключён по умолчанию и включается java -ea. Он подходит для внутренних предположений, но не заменяет проверку входа публичного метода. Для учебных обязательных тестов используйте явное выбрасывание AssertionError: тогда забытый -ea не превращает красный тест в зелёный.

```
public class Checks {
    static void equal(long expected, long actual) {
        if (expected != actual) {
            throw new AssertionError("expected=" + expected
                + ", actual=" + actual);
        }
    }
    public static void main(String[] args) {
        equal(6, Gcd.gcd(48, 18));
        equal(0, Gcd.gcd(0, 0));
        equal(2147483648L, Gcd.gcd(Integer.MIN_VALUE, 0));
        System.out.println("OK: 3 checks");
    }
}

javac --release 21 -g -d out examples/Gcd.java examples/Checks.java
java -cp out Checks
```

Разбор сеанса jdb

```
jdb -classpath out Gcd
stop in Gcd.gcd
run
locals
next
locals
where
print x
step
cont
exit
```

Первую строку вводите в Bash, остальные - в приглашении jdb. Точка останова устанавливается до run и может быть отложенной до загрузки класса. При входе в gcd доступны параметры x,y; переменные a,b появятся после выполнения объявлений. next выполняет строку, не заходя в вызванный метод; step может войти внутрь. where показывает цепочку кадров. cont продолжает до следующего события, в том числе второго вызова gcd.

Для расследования замените Math.abs((long)x) на Math.abs(x) в отдельной ветке, заново скомпилируйте с -g и запустите проверку MIN_VALUE. В jdb сравните x и a после присваивания. Причина ошибки не в остатке, а в типе перегрузки abs: int-версия уже вернула отрицательное значение до расширения. После исправления повторите регрессионный тест и сохраните его вместе с исправлением.

Стек исключения читают от места возникновения к вызывающим методам. Сначала воспроизведите проблему минимальным входом, затем сформулируйте гипотезу и проверьте одну переменную. Бессистемное добавление println может менять наблюдаемое поведение в сложных программах; в этом модуле отладчик нужен прежде всего для восстановления причинной цепочки.

Практика и самопроверка

7.1. База: заставьте Checks упасть и убедитесь, что процесс завершился ненулевым кодом.

7.2. Средний: остановитесь в gcd, запишите параметры, локальные переменные и стек на двух шагах.

7.3. Сложный: внесите ошибку right=mid-1 в lowerBound и найдите минимальный контрпример автоматическим перебором.

Контрольные вопросы: Почему три теста не доказывают корректность? Когда locals ещё не показывает переменную? Что отличает регрессионный тест?

Лабораторная работа: Сдайте расследование одной ошибки: симптом, минимальный вход, гипотеза, снимок состояния jdb, причина, исправление, проверка против повторения.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Занятие 8. Упаковка и воспроизводимость

JAR - архив с классами и ресурсами. Исполняемый JAR содержит атрибут Main-Class в манифесте; это не превращает архив в независимый от JVM бинарный файл. classpath нужен и при компиляции зависимых классов, и при запуске. Для Linux разделитель нескольких корней :; для Windows ;. В этом учебнике все команды рассчитаны на Linux.

```
javac --release 21 -encoding UTF-8 -g -d out src/course/Main.java
jar --create --file module1.jar --main-class course.Main -C out .
jar --list --file module1.jar
java -jar module1.jar
java -cp module1.jar course.Main
```

-C out . добавляет содержимое out в корень архива. Если добавить сам каталог out, ожидаемая структура course/Main.class окажется вложена на лишний уровень. java -jar использует сведения архива; не рассчитывайте, что обычный -cp будет определять его зависимости. В рамках модуля архив самодостаточен относительно ваших классов и требует установленной совместимой JVM.

Сборка как проверяемая последовательность

```
set -eu
mkdir -p build/classes
find src -name "*.java" -print > build/sources.txt
javac --release 21 -encoding UTF-8 -g -Xlint:all \
  -d build/classes @build/sources.txt
jar --create --file build/module1.jar \
  --main-class course.Main -C build/classes .
java -jar build/module1.jar
```

Эту последовательность сохраните в build.sh с первой строкой #!/usr/bin/env bash. Для данного файла со списком исходников договоримся, что пути не содержат пробелов: формат аргументного файла javac требует отдельного экранирования таких путей. Перед релизной проверкой используйте новый пустой каталог классов: старые .class могут скрывать отсутствие исходника. set -e останавливает обычную последовательность при ошибке; его поведение в условиях и конвейерах сложнее, поэтому не считайте его универсальной обработкой ошибок.

```
git switch -c feature/packaging
git add build.sh README.md
git diff --cached
git commit -m "Add documented CLI build"
git switch -
git merge feature/packaging
git log --oneline --graph --all
```

Ветка - подвижная ссылка на коммит. merge объединяет истории, а не просто копирует каталог. При конфликте прочитайте обе версии, отредактируйте итог, удалите маркеры, добавьте файл в индекс и завершите merge коммитом. Перед интеграцией убедитесь, что рабочие изменения сохранены. Для отмены уже опубликованного изменения обычно создают обратный коммит git revert; переписывание истории здесь не требуется.

Практика и самопроверка

8.1. База: соберите и запустите JAR, проверьте список записей.

8.2. Средний: добавьте ресурс и объясните различие ресурса в архиве и файла относительно текущего каталога; исследуйте getResourceAsStream.

8.3. Сложный: воспроизведите конфликт двух веток в README и разрешите его, сохранив смысл обеих правок.

Контрольные вопросы: Почему JAR требует JVM? Как старый class скрывает ошибку сборки? Что означает воспроизводимая проверка из чистого каталога?

Лабораторная работа: Передайте проект в новый каталог через git clone локального репозитория. Соберите только по README без копирования build/out. Сравните функциональный вывод; побайтовая идентичность JAR отдельно не гарантируется из-за метаданных.

Приёмка: приложите исходники, команды запуска, фактический вывод и объяснение граничных случаев. Каждый вывод о корректности подкрепите проверкой или рассуждением. Не ограничивайтесь одним успешным запуском.

Итоговая лабораторная. CLI MathLab

Создайте консольный инструмент course.MathLab с командами gcd, lcm, search и sqrt. Используйте только стандартную библиотеку JDK, собственные алгоритмы и сборку javac/jar. Не переносите сюда готовые решения из внешних источников. Примеры учебника являются опорой, но интерфейс, проверку входа и тесты вы пишете самостоятельно.

Контракт интерфейса

```
java -jar mathlab.jar gcd -48 18
# stdout: 6
java -jar mathlab.jar lcm 0 9
# stdout: 0
java -jar mathlab.jar search 3 1 3 3 8
# stdout: 1
java -jar mathlab.jar sqrt 2 1e-10
# stdout: приближение корня, одна строка
```

gcd и lcm принимают ровно два int, выдают неотрицательный long; gcd(0,0)=0. search принимает ключ int и далее ноль или больше отсортированных int, выдаёт lowerBound. Несортированный массив считается ошибкой входа. sqrt принимает конечное x от 0 до 1e12 и конечное eps от 1e-12 до 1e-3, возвращает приближение с абсолютной ошибкой не больше eps; для x=0 результат 0. Эти ограничения позволяют разумно анализировать округление, но не отменяют защиту от стагнации.

Успех: код завершения 0, ответ только в stdout. Ошибка команды, числа, количества аргументов или предусловия: код 2, объяснение в stderr, stdout пуст. Ожидаемые ошибки перехватывайте на уровне main. Непредвиденные дефекты не маскируйте под неверный ввод. Любой результат должен быть независим от текущего каталога, кроме явно используемых файлов проекта.

Этапы выполнения

Этап А, 3 часа: запишите контракты и таблицу примеров до кода. Создайте структуру src/course, test/course, README, .gitignore и журнал решений. Реализуйте разбор аргументов, не смешивая его с вычислением. Коммит должен собираться.

Этап Б, 5 часов: реализуйте вычислительные методы. Для каждого запишите инвариант, меру завершения и оценку сложности. sqrt строит начальный интервал [0,max(1,x)] и сохраняет корень внутри него; сравнение mid с x/mid позволяет избегать лишнего риска переполнения произведения при расширении диапазона.

Этап В, 4 часа: создайте явные проверки без внешнего фреймворка. Не менее 25 случаев, включая ошибки CLI; добавьте полный перебор малых входов для поиска и свойства gcd. Проверку stdout/stderr и кода возврата выполните из Bash. Намеренно испортите алгоритм и убедитесь, что тест это обнаружил.

Этап Г, 3 часа: соберите JAR из чистого каталога, пройдите сеанс jdb, оформите отчёт и защиту. Сдайте коммит или архив исходников, README с точными командами, протокол тестов и объяснение одного найденного дефекта. Не сдавайте только скриншоты и .class.

Сложные задачи со звёздочкой

S1. Расширенный алгоритм Евклида: для входных int найдите long-коэффициенты u,v с a*u+b*v=gcd(a,b). Отдельно исследуйте границы промежуточных значений и нулевые входы. Проверяйте тождество через BigInteger, чтобы тест не повторял переполнение реализации.

S2. Рациональная арифметика: реализуйте неизменяемую дробь с BigInteger, сложение, умножение, сравнение и сокращение. Докажите каноничность представления и согласованность compareTo с equals. Сравните цену операций с фиксированным long.

S3. Контрпример как продукт: для каждого из трёх неверных вариантов бинарного поиска найдите минимальный массив, докажите минимальность перебором и объясните, какой инвариант нарушен. Минимальность определяйте явно: сначала длина, затем лексикографический порядок.

S4. Исследуйте численную устойчивость корней квадратного уравнения: подберите коэффициенты с потерей точности, выведите устойчивую формулу и сравните с высокоточным эталоном. Эта задача дополнительная и не заменяет обязательную лабораторную.

Оценивание и защита

100 баллов: корректность алгоритмов и границы - 30; тесты и обнаружение дефектов - 20; доказательства и сложность - 15; интерфейс и ошибки - 10; Linux/Vim и самостоятельная сборка - 10; Git и история изменений - 5; читаемость, контракты и отчёт - 10. Дополнительные задачи отмечаются отдельно и не компенсируют отсутствие обязательной функциональности.

Зачёт: минимум 75/100, все четыре команды работают на базовых и граничных случаях, есть воспроизводимая сборка и тесты. 90+ означает уверенное выполнение с обоснованными ограничениями. Незавершённый проект не считается сданным независимо от красоты отчёта. При ошибке, нарушающей контракт, соответствующая часть отправляется на доработку.

Защита: объясните один участок байткода; покажите кадр gcd в jdb; докажите lowerBound; объясните MIN_VALUE; продемонстрируйте ошибку CLI с кодом 2; внесите небольшое изменение в Vim и сохраните отдельным коммитом. Разрешено пользоваться справкой инструментов; запрещено подменять объяснение заученным выводом.

Как отправлять работы в этот чат

Пишите идентификатор задания, прикладывайте исходники или текст кода, команды компиляции/запуска, ожидаемый и фактический результат. Укажите, что уже пробовали и где сомневаетесь. Для лабораторной приложите дерево проекта и отчёт тестов. Если присылаете изменённую версию, перечислите исправления.

Режим проверки: сначала я проверяю контракт и воспроизводимость, затем указываю минимальный контрпример и задаю направляющий вопрос. Первая подсказка указывает область проблемы; вторая помогает восстановить инвариант; третья даёт псевдокод отдельного фрагмента. Полное решение не выдаётся по умолчанию. После исправления выставляется оценка с замечаниями и следующий шаг. Готовое решение - только по вашей явной просьбе.

Статусы: не начато, в работе, нужны исправления, зачтено. Журнал в папке проекта служит сохраняемой опорой; обновляем его после проверки. Баллы не присваиваются за непроверенный код. Если контекст чата будет потерян, приложите журнал и последнюю версию работы.

Первое задание для сдачи

Начните с лабораторной занятия 1. Пришлите Hello.java, .gitignore, README, вывод java -version и javac -version, команды сборки, содержимое обоих файлов потоков и git log --oneline. Дополнительно объясните своими словами, что находится в индексе после изменения уже добавленного файла. Пока не переходите к итоговому проекту вместо базовой настройки.

Краткая памятка диагностики

Не компилируется: прочитайте первую ошибку javac, проверьте имя public-класса, пакет, типы, скобки и кодировку. Не запускается: проверьте java -version, двоичное имя и корень classpath. Неверный ответ: уменьшите вход, выпишите ожидаемые промежуточные значения, проверьте тип каждой операции. Падает тест: отделите неверное свойство теста от дефекта реализации. Потерялись изменения: сначала git status и git diff, затем решение; не применяйте команды удаления вслепую.

Справочники для углубления

Основной текст рассчитан на самостоятельное изучение без перехода по ссылкам. Спецификации помогают проверить спорную семантику. Команды javac и jdb сверены с официальной документацией Java 21; остальные ссылки являются справочными маршрутами.

https://docs.oracle.com/en/java/javase/21/docs/specs/man/javac.html

https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html

https://docs.oracle.com/en/java/javase/21/docs/specs/man/javap.html

https://docs.oracle.com/en/java/javase/21/docs/specs/man/jar.html

https://docs.oracle.com/en/java/javase/21/docs/specs/man/jdb.html

https://docs.oracle.com/javase/specs/jls/se21/html/index.html

https://docs.oracle.com/javase/specs/jvms/se21/html/index.html

https://git-scm.com/docs

https://vimhelp.org/